Print copy of official Microsoft Learn documentation

Source: https://learn.microsoft.com/en-us/dotnet/core/extensions/logging
Exported: 2026-10-14. Source metadata date: 2025-10-21T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

Logging in C# and .NET

.NET supports high performance, structured logging via the ILogger API to help monitor application behavior and diagnose issues. Configure different logging providers to write logs to different destinations. Basic logging providers are built-in, and many third-party providers are available.

Get started

This first example shows the basics, but it's only suitable for a trivial console app. This sample console app relies on the following NuGet packages:

- Microsoft.Extensions.Logging
- Microsoft.Extensions.Logging.Console

In the next section you see how to improve the code considering scale, performance, configuration, and typical programming patterns.

```
using Microsoft.Extensions.Logging;

using ILoggerFactory factory = LoggerFactory.Create(builder => builder.AddConsole());
ILogger logger = factory.CreateLogger("Program");
logger.LogInformation("Hello World! Logging is {Description}.", "fun");
```

The preceding example:

- Creates an ILoggerFactory. The `ILoggerFactory` stores all the configuration that determines where log messages are sent. In this case, configure the console logging provider so that log messages are written to the console.
- Creates an ILogger with a category named "Program". The category is a `string` that's associated with each message logged by the `ILogger` object. It groups log messages from the same class (or category) together when searching or filtering logs.
- Calls LogInformation to log a message at the `Information` level. The log level indicates the severity of the logged event and filters out less important log messages. The log entry also includes a message template `"Hello World! Logging is {Description}."` and a key-value pair `Description = fun`. The key name (or placeholder) comes from the word inside the curly braces in the template, and the value comes from the remaining method argument.

This project file for this example includes two NuGet packages:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Logging" Version="10.0.12" />
    <PackageReference Include="Microsoft.Extensions.Logging.Console" Version="10.0.12" />
  </ItemGroup>

</Project>
```

Tip

All of the logging example source code is available in the **Samples Browser** for download. For more information, see Browse code samples: Logging in .NET.

Logging in a non-trivial app

Consider making these changes to the previous example when logging in a less trivial scenario:

- If your application uses Dependency Injection (DI) or a host such as ASP.NET's WebApplication or Generic Host, use `ILoggerFactory` and `ILogger` objects from their respective DI containers rather than creating them directly. For more information, see Integration with DI and Hosts.
- Compile-time source generation for logging is usually a better alternative to `ILogger` extension methods like `LogInformation`. Source-generated logging offers better performance and stronger typing, and avoids spreading `string` constants throughout your methods. The tradeoff is that using this technique requires a bit more code.

  ```
  using Microsoft.Extensions.Logging;

  internal partial class Program
  {
      static void Main(string[] args)
      {
          using ILoggerFactory factory = LoggerFactory.Create(builder => builder.AddConsole());
          ILogger logger = factory.CreateLogger("Program");
          LogStartupMessage(logger, "fun");
      }

      [LoggerMessage(Level = LogLevel.Information, Message = "Hello World! Logging is
  {Description}.")]
      static partial void LogStartupMessage(ILogger logger, string description);
  }
  ```

- The recommended practice for choosing a log category name is to use the fully qualified name of the class that's creating the log message. This helps relate log messages back to the code that produced them and offers a good level of control when filtering logs. Specify the class type in the type parameter of the CreateLogger method.

```
using Microsoft.Extensions.Logging;

internal class Program
{
    static void Main(string[] args)
    {
        using ILoggerFactory factory = LoggerFactory.Create(builder => builder.AddConsole());
        ILogger logger = factory.CreateLogger<Program>();
        logger.LogInformation("Hello World! Logging is {Description}.", "fun");
    }
}
```

- If you don't use console logs as your sole production monitoring solution, add the logging providers you plan to use. For example, use OpenTelemetry to send logs over OTLP (OpenTelemetry protocol):

```
using Microsoft.Extensions.Logging;
using OpenTelemetry.Logs;

using ILoggerFactory factory = LoggerFactory.Create(builder =>
{
    builder.AddOpenTelemetry(logging =>
    {
        logging.AddOtlpExporter();
    });
});
ILogger logger = factory.CreateLogger("Program");
logger.LogInformation("Hello World! Logging is {Description}.", "fun");
```

Integration with hosts and dependency injection

If your application uses Dependency Injection (DI) or a host such as ASP.NET's WebApplication or Generic Host, use `ILoggerFactory` and `ILogger` objects from the DI container rather than creating them directly.

Get an ILogger from DI

This example gets an ILogger object in a hosted app using ASP.NET Minimal APIs:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<ExampleHandler>();

var app = builder.Build();

var handler = app.Services.GetRequiredService<ExampleHandler>();
app.MapGet("/", handler.HandleRequest);

app.Run();

partial class ExampleHandler(ILogger<ExampleHandler> logger)
{
    public string HandleRequest()
```

```
    {
        LogHandleRequest(logger);
        return "Hello World";
    }

    [LoggerMessage(LogLevel.Information, "ExampleHandler.HandleRequest was called")]
    public static partial void LogHandleRequest(ILogger logger);
}
```

The preceding example:

- Created a singleton service called `ExampleHandler` and mapped incoming web requests to run the `ExampleHandler.HandleRequest` function.
- Line 12 defines a primary constructor for the ExampleHandler, a feature added in C# 12. Using the older style C# constructor works equally well but is a little more verbose.
- The constructor defines a parameter of type `ILogger<ExampleHandler>`. ILogger<TCategoryName> derives from ILogger and indicates which category the `ILogger` object has. The DI container locates an `ILogger` with the correct category and supplies it as the constructor argument. If no `ILogger` with that category exists yet, the DI container automatically creates it from the `ILoggerFactory` in the service provider.
- The `logger` parameter received in the constructor is used for logging in the `HandleRequest` function.

Host-provided ILoggerFactory

Host builders initialize default configuration, then add a configured `ILoggerFactory` object to the host's DI container when the host is built. Before the host is built, adjust the logging configuration via HostApplicationBuilder.Logging, WebApplicationBuilder.Logging, or similar APIs on other hosts. Hosts also apply logging configuration from default configuration sources like *appsettings.json* and environment variables. For more information, see Configuration in .NET.

This example expands on the previous one to customize the `ILoggerFactory` provided by `WebApplicationBuilder`. It adds OpenTelemetry as a logging provider transmitting the logs over OTLP (OpenTelemetry protocol):

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Logging.AddOpenTelemetry(logging => logging.AddOtlpExporter());
builder.Services.AddSingleton<ExampleHandler>();
WebApplication app = builder.Build();
```

Create an ILoggerFactory with DI

If you're using a DI container without a host, use AddLogging to configure and add `ILoggerFactory` to the container.

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

// Add services to the container including logging
var services = new ServiceCollection();
services.AddLogging(builder => builder.AddConsole());
services.AddSingleton<ExampleService>();
IServiceProvider serviceProvider = services.BuildServiceProvider();
```

```
// Get the ExampleService object from the container
ExampleService service = serviceProvider.GetRequiredService<ExampleService>();

// Do some pretend work
service.DoSomeWork(10, 20);

class ExampleService(ILogger<ExampleService> logger)
{
    public void DoSomeWork(int x, int y)
    {
        logger.LogInformation("DoSomeWork was called. x={X}, y={Y}", x, y);
    }
}
```

The preceding example:

- Created a DI service container containing an `ILoggerFactory` configured to write to the console
- Added a singleton `ExampleService` to the container
- Created an instance of the `ExampleService` from the DI container, which also automatically created an `ILogger<ExampleService>` to use as the constructor argument.
- Invoked `ExampleService.DoSomeWork`, which used the `ILogger<ExampleService>` to log a message to the console.

Configure logging

Set logging configuration in code or via external sources, such as config files and environment variables. Using external configuration is beneficial when possible because you can change it without rebuilding the application. However, some tasks, such as setting logging providers, can only be configured from code.

Configure logging without code

For apps that use a host, the `"Logging"` section of *appsettings.`{Environment}`.json* files commonly provides logging configuration. For apps that don't use a host, set up external configuration sources explicitly or configure them in code instead.

The following *appsettings.Development.json* file is generated by the .NET Worker service templates:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  }
}
```

In the preceding JSON:

- The `"Default"`, `"Microsoft"`, and `"Microsoft.Hosting.Lifetime"` log level categories are specified.
- The `"Default"` value applies to all categories that aren't otherwise specified, effectively making all default values for all categories `"Information"`. Override this behavior by specifying a value for a

category.
- The "Microsoft" category applies to all categories that start with "Microsoft".
- The "Microsoft" category logs at a log level of Warning and higher.
- The "Microsoft.Hosting.Lifetime" category is more specific than the "Microsoft" category, so the "Microsoft.Hosting.Lifetime" category logs at log level "Information" and higher.
- A specific log provider isn't specified, so LogLevel applies to all the enabled logging providers except for the Windows EventLog.

The Logging property can have LogLevel and log provider properties. The LogLevel specifies the minimum level to log for selected categories. In the preceding JSON, Information and Warning log levels are specified. LogLevel indicates the severity of the log and ranges from 0 to 6:

Trace = 0, Debug = 1, Information = 2, Warning = 3, Error = 4, Critical = 5, and None = 6.

When a LogLevel is specified, logging is enabled for messages at the specified level and higher. In the preceding JSON, the Default category is logged for Information and higher. For example, Information, Warning, Error, and Critical messages are logged. If no LogLevel is specified, logging defaults to the Information level. For more information, see Log levels.

A provider property can specify a LogLevel property. LogLevel under a provider specifies levels to log for that provider, and overrides the non-provider log settings. Consider the following *appsettings.json* file:

```
{
    "Logging": {
        "LogLevel": {
            "Default": "Error",
            "Microsoft": "Warning"
        },
        "Debug": {
            "LogLevel": {
                "Default": "Information",
                "Microsoft.Hosting": "Trace"
            }
        },
        "EventSource": {
            "LogLevel": {
                "Default": "Warning"
            }
        }
    }
}
```

Settings in Logging.{ProviderName}.LogLevel override settings in Logging.LogLevel. In the preceding JSON, the Debug provider's default log level is set to Information:

Logging:Debug:LogLevel:Default:Information

The preceding setting specifies the Information log level for every Logging:Debug: category except Microsoft.Hosting. When a specific category is listed, the specific category overrides the default category. In the preceding JSON, the Logging:Debug:LogLevel categories "Microsoft.Hosting" and "Default" override the settings in Logging:LogLevel.

Specify the minimum log level for any of these:

- Specific providers: For example, `Logging:EventSource:LogLevel:Default:Information`
- Specific categories: For example, `Logging:LogLevel:Microsoft:Warning`
- All providers and all categories: `Logging:LogLevel:Default:Warning`

Any logs below the minimum level ***aren't***:

- Passed to the provider.
- Logged or displayed.

To suppress all logs, specify LogLevel.None. `LogLevel.None` has a value of 6, which is higher than `LogLevel.Critical` (5).

If a provider supports log scopes, `IncludeScopes` indicates whether they're enabled. For more information, see log scopes.

The following *appsettings.json* file contains settings for all of the built-in providers:

```
{
    "Logging": {
        "LogLevel": {
            "Default": "Error",
            "Microsoft": "Warning",
            "Microsoft.Hosting.Lifetime": "Warning"
        },
        "Debug": {
            "LogLevel": {
                "Default": "Information"
            }
        },
        "Console": {
            "IncludeScopes": true,
            "LogLevel": {
                "Microsoft.Extensions.Hosting": "Warning",
                "Default": "Information"
            }
        },
        "EventSource": {
            "LogLevel": {
                "Microsoft": "Information"
            }
        },
        "EventLog": {
            "LogLevel": {
                "Microsoft": "Information"
            }
        },
        "AzureAppServicesFile": {
            "IncludeScopes": true,
            "LogLevel": {
                "Default": "Warning"
            }
        },
        "AzureAppServicesBlob": {
            "IncludeScopes": true,
            "LogLevel": {
                "Microsoft": "Information"
            }
        },
        "ApplicationInsights": {
            "LogLevel": {
```

```
                "Default": "Information"
            }
        }
    }
}
```

In the preceding sample:

- The categories and levels aren't suggested values. The sample shows all the default providers.
- Settings in `Logging.{ProviderName}.LogLevel` override settings in `Logging.LogLevel`. For example, the level in `Debug.LogLevel.Default` overrides the level in `LogLevel.Default`.
- Each provider's *alias* is used. Each provider defines an *alias* that you can use in configuration in place of the fully qualified type name. The built-in providers' aliases are:
    - `Console`
    - `Debug`
    - `EventSource`
    - `EventLog`
    - `AzureAppServicesFile`
    - `AzureAppServicesBlob`
    - `ApplicationInsights`

Set log level by command line, environment variables, and other configuration

Set the log level using any of the configuration providers. For example, create a persisted environment variable named `Logging:LogLevel:Microsoft` with a value of `Information`.

- Command Line
- PowerShell
- Bash

Create and assign persisted environment variable, given the log level value.

```
:: Assigns the env var to the value
setx "Logging__LogLevel__Microsoft" "Information" /M
```

In a *new* instance of the **Command Prompt**, read the environment variable.

```
:: Prints the env var value
echo %Logging__LogLevel__Microsoft%
```

The preceding environment setting persists in the environment. To test the settings when using an app created with the .NET Worker service templates, use the `dotnet run` command in the project directory after the environment variable is assigned.

```
dotnet run
```

Tip

After setting an environment variable, restart your integrated development environment (IDE) to ensure that newly added environment variables are available.

On Azure App Service, select **New application setting** on the **Settings > Configuration** page. Azure App Service application settings are:

- Encrypted at rest and transmitted over an encrypted channel.
- Exposed as environment variables.

For more information on setting .NET configuration values using environment variables, see environment variables.

Configure logging with code

To configure logging in code, use the ILoggingBuilder API. You can access it from different places:

- When creating the `ILoggerFactory` directly, configure in LoggerFactory.Create.
- When using DI without a host, configure in LoggingServiceCollectionExtensions.AddLogging.
- When using a host, configure with HostApplicationBuilder.Logging, WebApplicationBuilder.Logging or other host specific APIs.

This example shows setting the console logging provider and several filters.

```
using Microsoft.Extensions.Logging;

using var loggerFactory = LoggerFactory.Create(static builder =>
{
    builder
        .AddFilter("Microsoft", LogLevel.Warning)
        .AddFilter("System", LogLevel.Warning)
        .AddFilter("LoggingConsoleApp.Program", LogLevel.Debug)
        .AddConsole();
});

ILogger logger = loggerFactory.CreateLogger<Program>();
logger.LogDebug("Hello {Target}", "Everyone");
```

In the preceding example, AddFilter adjusts the log level that's enabled for various categories. AddConsole adds the console logging provider. By default, logs with `Debug` severity aren't enabled, but because the configuration adjusted the filters, the debug message "Hello Everyone" is displayed on the console.

How filtering rules are applied

When an ILogger<TCategoryName> object is created, the ILoggerFactory object selects a single rule per provider to apply to that logger. The `ILogger` instance filters all messages it writes based on the selected rules. The most specific rule for each provider and category pair is selected from the available rules.

The following algorithm is used for each provider when an `ILogger` is created for a given category:

- Select all rules that match the provider or its alias. If no match is found, select all rules with an empty provider.

- From the result of the preceding step, select rules with longest matching category prefix. If no match is found, select all rules that don't specify a category.
- If multiple rules are selected, take the **last** one.
- If no rules are selected, use LoggingBuilderExtensions.SetMinimumLevel(ILoggingBuilder, LogLevel) to specify the minimum logging level.

Log category

When an `ILogger` object is created, a *category* is specified. That category is included with each log message created by that instance of `ILogger`. The category string is arbitrary, but the convention is to use the fully qualified class name. For example, in an application with a service defined like the following object, the category might be `"Example.DefaultService"`:

```
namespace Example
{
    public class DefaultService : IService
    {
        private readonly ILogger<DefaultService> _logger;

        public DefaultService(ILogger<DefaultService> logger) =>
            _logger = logger;

        // ...
    }
}
```

If further categorization is desired, the convention is to use a hierarchical name by appending a subcategory to the fully qualified class name, and explicitly specify the category using LoggerFactory.CreateLogger:

```
namespace Example
{
    public class DefaultService : IService
    {
        private readonly ILogger _logger;

        public DefaultService(ILoggerFactory loggerFactory) =>
            _logger = loggerFactory.CreateLogger("Example.DefaultService.CustomCategory");

        // ...
    }
}
```

Calling `CreateLogger` with a fixed name is useful when used in multiple classes/types so the events can be organized by category.

`ILogger<T>` is equivalent to calling `CreateLogger` with the fully qualified type name of `T`.

Log level

The following table lists the LogLevel values, the convenience `Log{LogLevel}` extension method, and the suggested usage:

LogLevel	Value	Method	Description
Trace	0	LogTrace	Contain the most detailed messages. These messages might contain sensitive app data. These messages are disabled by default and should ***not*** be enabled in production.
Debug	1	LogDebug	For debugging and development. Use with caution in production due to the high volume.
Information	2	LogInformation	Tracks the general flow of the app. Might have long-term value.
Warning	3	LogWarning	For abnormal or unexpected events. Typically includes errors or conditions that don't cause the app to fail.
Error	4	LogError	For errors and exceptions that can't be handled. These messages indicate a failure in the current operation or request, not an app-wide failure.
Critical	5	LogCritical	For failures that require immediate attention. Examples: data loss scenarios, out of disk space.
None	6		Specifies that no messages should be written.

In the previous table, the `LogLevel` is listed from lowest to highest severity.

The Log method's first parameter, LogLevel, indicates the severity of the log. Rather than calling `Log(LogLevel, ...)`, most developers call the Log{LogLevel} extension methods. The `Log{LogLevel}` extension methods call the `Log` method and specify the `LogLevel`. For example, the following two logging calls are functionally equivalent and produce the same log:

```
public void LogDetails()
{
    var logMessage = "Details for log.";

    _logger.Log(LogLevel.Information, AppLogEvents.Details, logMessage);
    _logger.LogInformation(AppLogEvents.Details, logMessage);
}
```

`AppLogEvents.Details` is the event ID, and is implicitly represented by a constant Int32 value. `AppLogEvents` is a class that exposes various named identifier constants and is displayed in the Log event ID section.

The following code creates `Information` and `Warning` logs:

```
public async Task<T> GetAsync<T>(string id)
{
    _logger.LogInformation(AppLogEvents.Read, "Reading value for {Id}", id);

    var result = await _repository.GetAsync(id);
    if (result is null)
    {
        _logger.LogWarning(AppLogEvents.ReadNotFound, "GetAsync({Id}) not found", id);
    }

    return result;
}
```

In the preceding code, the first `Log{LogLevel}` parameter, `AppLogEvents.Read`, is the Log event ID. The second parameter is a message template with placeholders for argument values provided by the remaining method parameters. The method parameters are explained in the message template section later in this article.

Configure the appropriate log level and call the correct `Log{LogLevel}` methods to control how much log output is written to a particular storage medium. For example:

- In production:
  - Logging at the `Trace` or `Debug` levels produces a high-volume of detailed log messages. To control costs and not exceed data storage limits, log `Trace` and `Debug` level messages to a high-volume, low-cost data store. Consider limiting `Trace` and `Debug` to specific categories.
  - Logging at `Warning` through `Critical` levels should produce few log messages.
    - Costs and storage limits usually aren't a concern.
    - Few logs allow more flexibility in data store choices.
- In development:
  - Set to `Warning`.
  - Add `Trace` or `Debug` messages when troubleshooting. To limit output, set `Trace` or `Debug` only for the categories under investigation.

The following JSON sets `Logging:Console:LogLevel:Microsoft:Information`:

```
{
    "Logging": {
        "LogLevel": {
            "Microsoft": "Warning"
        },
        "Console": {
            "LogLevel": {
                "Microsoft": "Information"
            }
        }
    }
}
```

Log event ID

Each log can specify an *event identifier*, the EventId is a structure with an `Id` and optional `Name` readonly properties. The sample source code uses the `AppLogEvents` class to define event IDs:

```
using Microsoft.Extensions.Logging;

internal static class AppLogEvents
{
    internal static EventId Create = new(1000, "Created");
    internal static EventId Read = new(1001, "Read");
    internal static EventId Update = new(1002, "Updated");
    internal static EventId Delete = new(1003, "Deleted");

    // These are also valid EventId instances, as there's
    // an implicit conversion from int to an EventId
    internal const int Details = 3000;
    internal const int Error = 3001;
```

```
    internal static EventId ReadNotFound = 4000;
    internal static EventId UpdateNotFound = 4001;

    // ...
}
```

> Tip
>
> For more information on converting an `int` to an `EventId`, see EventId.Implicit(Int32 to EventId) Operator.

An event ID associates a set of events. For example, all logs related to reading values from a repository might be `1001`.

The logging provider might log the event ID in an ID field, in the logging message, or not at all. The Debug provider doesn't show event IDs. The console provider shows event IDs in brackets after the category:

```
info: Example.DefaultService.GetAsync[1001]
      Reading value for a1b2c3
warn: Example.DefaultService.GetAsync[4000]
      GetAsync(a1b2c3) not found
```

Some logging providers store the event ID in a field, which allows for filtering on the ID.

Log message template

Each log API uses a message template. The message template can contain placeholders for which arguments are provided. Use names for the placeholders, not numbers. The order of placeholders, not their names, determines which parameters are used to provide their values. In the following code, the parameter names are out of sequence in the message template:

```
string p1 = "param1";
string p2 = "param2";
_logger.LogInformation("Parameter values: {p2}, {p1}", p1, p2);
```

The preceding code creates a log message with the parameter values in sequence:

```
Parameter values: param1, param2
```

> Note
>
> Be mindful when using multiple placeholders within a single message template, as they're ordinal-based. The names aren't used to align the arguments to the placeholders.

This approach lets logging providers implement semantic or structured logging. The arguments themselves are passed to the logging system, not just the formatted message template. This enables logging providers to store the parameter values as fields. Consider the following logger method:

```
_logger.LogInformation("Getting item {Id} at {RunTime}", id, DateTime.Now);
```

For example, when logging to Azure Table Storage:

- Each Azure Table entity can have `ID` and `RunTime` properties.
- Tables with properties simplify queries on logged data. For example, a query can find all logs within a particular `RunTime` range without having to parse the time out of the text message.

Log message template formatting

Log message templates support placeholder formatting. Templates can specify any valid format for the given type argument. For example, consider the following `Information` logger message template:

```
_logger.LogInformation("Logged on {PlaceHolderName:MMMM dd, yyyy}", DateTimeOffset.UtcNow);
// Logged on January 06, 2022
```

In the preceding example, the `DateTimeOffset` instance is the type that corresponds to the `PlaceHolderName` in the logger message template. This name can be anything as the values are ordinal-based. The `MMMM dd, yyyy` format is valid for the `DateTimeOffset` type.

For more information on `DateTime` and `DateTimeOffset` formatting, see Custom date and time format strings.

Examples

The following examples show how to format a message template using the `{}` placeholder syntax. Additionally, an example of escaping the `{}` placeholder syntax is shown with its output. Finally, string interpolation with templating placeholders is also shown:

```
logger.LogInformation("Number: {Number}", 1);               // Number: 1
logger.LogInformation("{{Number}}: {Number}", 3);           // {Number}: 3
logger.LogInformation($"{{{{Number}}}}: {{Number}}", 5);    // {Number}: 5
```

> Tip
>
> - In most cases, you should use log message template formatting when logging. Use of string interpolation can cause performance issues.
> - Code analysis rule CA2254: Template should be a static expression helps alert you to places where your log messages don't use proper formatting.

Log exceptions

The logger methods have overloads that take an exception parameter:

```
public void Test(string id)
{
    try
```

```
    {
        if (id is "none")
        {
            throw new Exception("Default Id detected.");
        }
    }
    catch (Exception ex)
    {
        _logger.LogWarning(
            AppLogEvents.Error, ex,
            "Failed to process iteration: {Id}", id);
    }
}
```

Exception logging is provider-specific.

Default log level

If the default log level isn't set, the default log level value is `Information`.

For example, consider the following worker service app:

- Created with the .NET Worker templates.
- *appsettings.json* and *appsettings.Development.json* deleted or renamed.

With the preceding setup, navigating to the privacy or home page produces many `Trace`, `Debug`, and `Information` messages with `Microsoft` in the category name.

The following code sets the default log level when the default log level isn't set in configuration:

```
HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

builder.Logging.SetMinimumLevel(LogLevel.Warning);

using IHost host = builder.Build();

await host.RunAsync();
```

Filter function

A filter function is invoked for all providers and categories that don't have rules assigned to them by configuration or code:

```
HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

builder.Logging.AddFilter((provider, category, logLevel) =>
{
    return provider.Contains("ConsoleLoggerProvider")
        && (category.Contains("Example") || category.Contains("Microsoft"))
        && logLevel >= LogLevel.Information;
});

using IHost host = builder.Build();

await host.RunAsync();
```

The preceding code displays console logs when the category contains `Example` or `Microsoft` and the log level is `Information` or higher.

Log scopes

A *scope* groups a set of logical operations. This grouping can attach the same data to each log that's created as part of a set. For example, every log created as part of processing a transaction can include the transaction ID.

A scope:

- Is an IDisposable type that's returned by the BeginScope method.
- Lasts until it's disposed.

The following providers support scopes:

- `Console`
- AzureAppServicesFile and AzureAppServicesBlob
- ApplicationInsightsLoggerProvider

Use a scope by wrapping logger calls in a `using` block:

```
public async Task<T> GetAsync<T>(string id)
{
    T result;
    var transactionId = Guid.NewGuid().ToString();

    using (_logger.BeginScope(new List<KeyValuePair<string, object>>
        {
            new KeyValuePair<string, object>("TransactionId", transactionId),
        }))
    {
        _logger.LogInformation(
            AppLogEvents.Read, "Reading value for {Id}", id);

        var result = await _repository.GetAsync(id);
        if (result is null)
        {
            _logger.LogWarning(
                AppLogEvents.ReadNotFound, "GetAsync({Id}) not found", id);
        }
    }

    return result;
}
```

The following JSON enables scopes for the console provider:

```
{
    "Logging": {
        "Debug": {
            "LogLevel": {
                "Default": "Information"
            }
        },
        "Console": {
```

```
            "IncludeScopes": true,
            "LogLevel": {
                "Microsoft": "Warning",
                "Default": "Information"
            }
        },
        "LogLevel": {
            "Default": "Debug"
        }
    }
}
```

The following code enables scopes for the console provider:

```
HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

builder.Logging.ClearProviders();
builder.Logging.AddSimpleConsole(options => options.IncludeScopes = true);

using IHost host = builder.Build();

await host.RunAsync();
```

Create logs in Main

The following code logs in `Main` by getting an `ILogger` instance from DI after building the host:

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;

using IHost host = Host.CreateApplicationBuilder(args).Build();

var logger = host.Services.GetRequiredService<ILogger<Program>>();
logger.LogInformation("Host created.");

await host.RunAsync();
```

The preceding code relies on two NuGet packages:

- Microsoft.Extensions.Hosting
- Microsoft.Extensions.Logging

Its project file looks similar to the following:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net7.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="7.0.1" />
    <PackageReference Include="Microsoft.Extensions.Logging" Version="7.0.0" />
```

```
  </ItemGroup>

</Project>
```

No asynchronous logger methods

Logging should be so fast that it isn't worth the performance cost of asynchronous code. If a logging datastore is slow, don't write to it directly. Consider writing the log messages to a fast store initially, then moving them to the slow store later. For example, when logging to SQL Server, don't do so directly in a `Log` method, since the `Log` methods are synchronous. Instead, synchronously add log messages to an in-memory queue and have a background worker pull the messages out of the queue to do the asynchronous work of pushing data to SQL Server.

Change log levels in a running app

The Logging API doesn't include a scenario to change log levels while an app is running. However, some configuration providers can reload configuration, which takes immediate effect on logging configuration. For example, the File Configuration Provider reloads logging configuration by default. If you change the configuration in code while an app is running, the app can call IConfigurationRoot.Reload to update the app's logging configuration.

NuGet packages

The ILogger<TCategoryName> and ILoggerFactory interfaces and implementations are included in most .NET SDKs as implicit package reference. They're also available explicitly in the following NuGet packages when not otherwise implicitly referenced:

- The interfaces are in Microsoft.Extensions.Logging.Abstractions.
- The default implementations are in Microsoft.Extensions.Logging.

For more information about which .NET SDK includes implicit package references, see .NET SDK: table to implicit namespace.

See also

- Logging providers in .NET
- Implement a custom logging provider in .NET
- Console log formatting
- High-performance logging in .NET
- Logging guidance for .NET library authors
- Logging bugs should be created in the github.com/dotnet/runtime repo